EVA-ML-Overview

📄 EVA Product Requirements Document

Machine Learning Reasoning Layer (MLRL)

---

1. Purpose

The **Machine Learning Reasoning Layer (MLRL)** provides EVA with the ability to learn predictive patterns from data when analytical reasoning alone is insufficient to answer the user's objective.

EVA is fundamentally an analytical reasoning system.
Machine learning is a supporting capability, not the primary intelligence.

The MLRL is invoked only when the Global Analysis Ledger determines that:

- a future outcome must be predicted
- a classification decision is required
- or pattern generalization cannot be achieved through descriptive analysis

If these conditions are not met, EVA completes the analysis without building a model.

---

2. Design Philosophy

Traditional ML systems begin with models and search for meaning later.

EVA follows the reverse:

understanding → reasoning → hypothesis → evidence → **then learning**

Machine learning in EVA is therefore:

- contextual
- explainable
- evidence-bound

The model is treated as an instrument used by the analyst, not as the analyst itself.

Key rule:

EVA never trusts a model more than it trusts the evidence already observed in the dataset.

---

3. When Machine Learning Is Activated

The MLRL activates only after completion of:

• Dataset Profiler & Semantic Understanding
• Question Builder / Intent Inference
• Data Repair & Integrity Layer
• Feature Intelligence Engine
• Investigation & Hypothesis Engine

Machine learning is triggered when the User Intent Record specifies:

Prediction
Forecasting
Automated classification
Risk scoring
Detection of unseen cases

Machine learning is not activated for:

Pure explanation
Simple segmentation
Descriptive reporting
Trend visualization

---

4. Relationship to the Global Analysis Ledger

The MLRL both reads from and writes to the Global Analysis Ledger (GAL).

It reads:

• selected target variable
• validated features
• restricted columns
• hypotheses
• stakeholder requirements
• interpretability priority

It writes:

• modeling problem definition
• training data construction record
• candidate model reasoning
• evaluation results
• reliability analysis
• prediction confidence

All model decisions must be reproducible using the GAL alone.

---

5. Machine Learning Pipeline Overview

The Machine Learning Reasoning Layer consists of six coordinated modules:

1. Problem Framing Module
2. Training Data Constructor
3. Model Candidate Generator
4. Training & Evaluation Module
5. Reliability & Validation (Explainability) Module
6. Prediction & Monitoring Module

Each module performs a reasoning task rather than a purely computational task.

---

6. Data Flow

The ML pipeline operates on a **Learning View**, not the raw dataset.

The Learning View is constructed from:

• repaired dataset
• engineered features
• permitted columns only

Flow:

Global Analysis Ledger
→ Problem Framing
→ Learning View Construction
→ Candidate Models
→ Training & Evaluation
→ Reliability Validation
→ Final Model

→ Prediction & Monitoring
→ Dashboard & Report

---

7. Model Selection Principles

EVA does not select a model based solely on performance score.

Selection must consider:

• interpretability needs
• dataset size
• stability across splits
• alignment with hypotheses
• risk of overfitting
• stakeholder type

A slightly less accurate but explainable model may be preferred.

---

8. Reliability Requirement

Every selected model must satisfy reliability checks:

• consistent performance across validation sets
• reasonable sensitivity to feature variation
• alignment with observed real-world patterns

If a model contradicts strong earlier evidence from the Investigation & Hypothesis Engine, EVA must flag the model for review instead of deploying it.

---

9. Dockerized Service Architecture

The Machine Learning Reasoning Layer is implemented as isolated services.

Each module operates as an independent service to ensure:

• reproducibility
• failure isolation
• re-execution of individual steps
• scalability

The services communicate through structured artifacts rather than direct memory sharing.

---

10. Service Responsibilities

Service	Responsibility
Problem Framer	Define modeling task and target
Data Constructor	Build learning dataset
Candidate Generator	Propose modeling approaches
Trainer	Train and evaluate candidates
Validator	Check reliability and explainability
Predictor	Serve predictions and monitor behavior

---

11. Session Artifact Storage

Each analysis session creates a dedicated directory.

Example structure:

```
eva-session-data/
  sessions/
    <session_id>/
      dataset_snapshot/
      repaired_dataset/
      feature_view/
      learning_view/
      model_candidates/
      evaluation_results/
      selected_model/
      prediction_logs/
      ledger_snapshot/
```

Artifacts must be preserved to ensure full reproducibility of analysis.

---

12. Model Artifacts

The selected model must be stored with:

• feature schema
• training metadata
• evaluation metrics
• reliability report

The model cannot be separated from its context.

---

13. Prediction Behavior

Predictions must include:

• predicted outcome
• confidence estimate
• risk level
• explanation reference

Predictions without confidence are not permitted.

---

14. Monitoring Responsibilities

After deployment, EVA monitors:

• prediction confidence drift
• data distribution changes
• unusual input patterns

If reliability decreases, EVA must notify the Dashboard Composer.

---

15. Interaction with Dashboard & Report

Dashboard Composer
→ shows high-risk predictions and monitored trends

Report Generator
→ explains how the model was used and its limitations

The model is never presented without context.

---

16. Operational Rules

1. ML cannot run without a defined target.
2. Restricted columns cannot be used.
3. All training steps must be recorded.
4. Models must pass reliability validation.
5. Predictions must include uncertainty.

---

17. Why This Layer Exists

The Machine Learning Reasoning Layer allows EVA to extend beyond describing reality into anticipating it.

EVA does not build models to showcase accuracy.
EVA builds models to support decisions.

The system therefore answers:

not only “what is happening”
but also
“what is likely to happen next.”

Problem Framing & Training Data Construction

📄 EVA System Module Specification

Problem Framing & Training Data Construction (PFTDC)

---

1. Purpose

The **Problem Framing & Training Data Construction (PFTDC)** module determines whether machine learning is appropriate for the current analysis session and, if so, defines the exact learning problem and constructs the dataset that the model will learn from.

This module converts analytical intent into a formal learning task.

Machine learning models do not fail primarily due to algorithms.
They fail because they are trained to solve the wrong problem.

The PFTDC module prevents incorrect modeling by ensuring that the prediction task is meaningful, well-defined, and grounded in the user's objective and the dataset's real-world interpretation.

---

2. Design Philosophy

A dataset does not automatically imply a prediction task.

Before any model is created, EVA must answer:

• What future or unknown outcome are we predicting?
• Does predicting it provide value to the user?
• Is the dataset capable of supporting such prediction?

If these questions cannot be answered confidently, the ML pipeline must not proceed.

The system prefers **no model** over a misleading model.

---

3. Inputs

The module reads from the Global Analysis Ledger:

• dataset identity and row meaning
• time behavior classification
• candidate target variables
• user intent and decision objective
• restricted feature list (leakage risks)
• hypotheses and key findings
• engineered feature set

It does not rely on raw dataset assumptions.

---

4. Determining Whether Modeling Is Needed

The module evaluates if a predictive task exists.

Machine learning is appropriate only if:

• the user needs a future outcome
• the outcome is not already known at record creation
• the dataset contains meaningful predictors

Machine learning is rejected when:

• the analysis goal is purely explanatory
• no valid target variable exists
• the dataset is too small or incomplete
• prediction would not influence a decision

If rejected, the module records the reason in the Global Analysis Ledger and ends the ML pipeline.

---

5. Problem Type Identification

If modeling is justified, the module classifies the learning problem.

Possible problem types:

Binary Classification
Predict yes/no outcome

Multi-Class Classification
Predict one category among many

Regression
Predict a continuous value

Time Forecasting
Predict a future value over time

Anomaly Detection
Detect unusual future behavior

Clustering (Fallback)
Group entities when no explicit outcome exists but decision value remains

The classification is recorded along with justification.

---

6. Target Variable Selection

The module confirms the target variable using:

• Dataset Profiler candidates
• User intent
• Real-world interpretation

Rules:

• The target must represent a future or unknown outcome
• The target must not contain information unavailable at prediction time
• The target must be decision-relevant

If multiple targets exist, the module prioritizes based on user objective.

---

7. Learning View Construction

The model will not train on the raw dataset.

Instead, the module constructs a **Learning View** — a structured dataset designed specifically for learning.

The Learning View is created using:

• repaired dataset
• validated engineered features
• permitted columns only

Excluded:

• identifier columns
• leakage columns
• post-outcome information

This step ensures the model learns only legitimate patterns.

---

8. Feature Eligibility Filtering

Every feature is evaluated before inclusion.

A feature is excluded if:

• it directly encodes the outcome
• it depends on future information
• it violates real-world availability at prediction time
• it contradicts identified misinterpretation risks

The module records accepted and rejected features in the ledger.

---

9. Handling Time-Dependent Data

If the dataset is temporal:

The module enforces chronological learning rules.

Training data must come strictly from earlier time periods than validation data.

Random mixing of time periods is forbidden because it creates unrealistic performance estimates.

The module records the time boundary used.

---

10. Data Splitting Strategy

The Learning View is divided into:

• training set
• validation set
• holdout test set

The split must:

• preserve class distribution when relevant
• respect time ordering when applicable
• avoid entity duplication across sets

The module records split reasoning to ensure reproducibility.

---

11. Minimum Viability Check

Before allowing training, the module verifies:

• sufficient number of records
• sufficient variation in target
• usable feature count
• no dominant missingness

If the dataset cannot support reliable learning, modeling is halted and logged.

---

12. Output Written to Global Analysis Ledger

The module writes the **Model Definition Record**, containing:

Modeling Decision
Whether ML is used and why

Problem Type
Classification, regression, etc.

Selected Target
Outcome variable

Learning View Schema
Features allowed

Excluded Features
Leakage and restricted columns

Split Strategy
How training/validation/test were created

Feasibility Assessment
Confidence in training viability

---

13. Operational Rules

1. No model may train without this module's approval.
2. Raw dataset must never be directly used for training.
3. All excluded columns must be recorded.
4. Time leakage must be prevented.
5. The model problem must match user intent.

---

14. Relationship to Other Modules

Feature Intelligence Engine
→ provides candidate features

Data Repair & Integrity Layer
→ provides clean dataset

Investigation & Hypothesis Engine
→ provides real-world context

Training & Evaluation Module
→ consumes the Learning View

Reliability & Validation Module
→ verifies alignment with hypotheses

---

15. Why This Module Is Critical

Most ML systems start with training.

EVA starts with **defining what it means to learn**.

If the problem is framed incorrectly, a high-accuracy model can still produce harmful decisions.

The Problem Framing & Training Data Construction module ensures that any model EVA builds is relevant, valid, and ethically defensible.

Model Candidate Generator

📄 EVA System Module Specification

Model Candidate Generator (MCG)

---

1. Purpose

The **Model Candidate Generator (MCG)** module determines which learning approaches EVA should consider for the defined prediction problem.

Instead of blindly testing many algorithms, this module reasons about what types of models are appropriate given:

• problem type
• dataset size
• feature characteristics
• user interpretability requirements
• stability needs

The module outputs a small, justified set of candidate modeling approaches for evaluation.

---

2. Design Philosophy

Not all models are suitable for all datasets.

A highly complex model may perform well on training data but produce unreliable or uninterpretable decisions.

EVA therefore prioritizes **appropriateness over complexity**.

The system must prefer:

a reliable understandable model
over
a marginally more accurate opaque model

The goal is decision support, not leaderboard performance.

---

3. Inputs

The module reads from the Global Analysis Ledger:

• problem type (classification, regression, forecasting, anomaly)
• learning view schema
• number of records
• feature count and feature types
• class distribution
• stakeholder type
• interpretability requirement
• hypothesis results

---

4. Candidate Selection Strategy

The module selects modeling families, not specific implementations.

Selection depends on four factors:

1. Data size
2. Feature structure
3. Noise level
4. Explanation requirement

The module creates a **balanced portfolio** of candidates including:

• interpretable baseline
• flexible learner
• robustness-oriented method

This ensures meaningful comparison rather than random experimentation.

---

5. Interpretable Baseline Requirement

Every modeling session must include at least one interpretable candidate.

Purpose:

• provide reference performance
• provide understandable relationships
• act as sanity check

If a complex model outperforms the baseline only slightly, the baseline may be preferred.

---

6. Complexity Control

The module avoids excessive complexity when:

• dataset is small
• features are few
• noise is high
• user requires explanation

The module increases flexibility when:

• large dataset exists
• complex patterns detected
• predictive accuracy is critical

---

7. Metric Selection

Evaluation metrics are chosen based on user intent.

Examples of priorities:

Risk detection → minimize missed risky cases
Decision automation → minimize false actions
Estimation tasks → minimize prediction deviation
Educational use → emphasize interpretability

The module records the chosen metric and justification.

---

8. Hypothesis Alignment Check

The module compares model expectations with previously generated hypotheses.

If hypotheses suggest strong linear relationships, simpler models are prioritized.

If relationships appear nonlinear or interaction-heavy, flexible candidates are included.

The module does not allow modeling assumptions that contradict observed data behavior.

---

9. Candidate Portfolio Output

The module outputs a **Candidate Set** consisting of:

• 2–4 modeling approaches
• reasoning for inclusion
• expected strengths
• expected weaknesses

This set is intentionally limited to ensure meaningful evaluation rather than exhaustive search.

---

10. Output Written to Global Analysis Ledger

The module writes the **Candidate Model Record**, containing:

Candidate List
Selected learning approaches

Reasoning
Why each candidate was chosen

Expected Behavior
What patterns each should capture

Chosen Evaluation Metric
How success will be measured

Interpretability Expectation
Level of explanation possible

---

11. Operational Rules

1. At least one interpretable candidate is mandatory.
2. Candidate count must remain limited.
3. Candidate reasoning must reference dataset characteristics.
4. Metric choice must align with user intent.
5. Candidates cannot contradict data understanding.

---

12. Relationship to Other Modules

Problem Framing & Training Data Construction
→ defines learning task

Training & Evaluation Module
→ trains candidates

Reliability & Validation Module
→ verifies behavior

Prediction & Monitoring Module
→ deploys selected model

---

13. Why This Module Is Critical

AutoML systems optimize performance.

EVA optimizes decisions.

Choosing models intelligently:

• reduces overfitting risk
• improves trust
• increases explainability
• prevents meaningless complexity

The Model Candidate Generator ensures EVA behaves like a careful ML practitioner rather than an automated experiment runner.

Training & Evaluation Module

📄 EVA System Module Specification

Training & Evaluation Module (TEM)

---

1. Purpose

The **Training & Evaluation Module (TEM)** trains the candidate learning approaches selected by the Model Candidate Generator and evaluates their performance, stability, and generalization reliability.

The module does not aim to maximize performance metrics alone.
Its objective is to determine which model can be safely trusted to support decisions.

The module produces model evidence rather than just a performance score.

---

2. Design Philosophy

High accuracy does not necessarily mean a useful model.

A model may:

• memorize training data
• exploit data leakage
• behave inconsistently
• fail under small changes

Therefore, EVA evaluates models on three criteria:

Performance
Stability
Generalization

The selected model must satisfy all three.

---

3. Inputs

The module reads from the Global Analysis Ledger:

• learning view dataset
• training/validation/test split
• candidate modeling approaches
• evaluation metric
• interpretability requirement

It does not modify the dataset.

---

4. Training Procedure

For each candidate:

1. Train on the training set
2. Evaluate on validation set
3. Perform repeat validation checks
4. Record behavior

Training must be reproducible using the same data and configuration.

---

5. Performance Evaluation

The module calculates performance using the metric chosen earlier.

Performance evaluation includes:

• validation performance
• holdout test performance
• comparison to baseline

A candidate is rejected if:

• performance is worse than baseline
• performance is inconsistent

---

6. Overfitting Detection

The module checks whether the model memorizes instead of learning.

Indicators include:

• large gap between training and validation performance
• sudden performance drop on unseen data

If detected, the model is marked unreliable.

---

7. Stability Testing

The module evaluates how sensitive the model is to small changes.

The system tests:

• different training splits
• small variations in input

A stable model should produce similar predictions across reasonable variations.

Highly unstable models are rejected even if accurate.

---

8. Generalization Assessment

The module evaluates how well the model performs on completely unseen data.

This is measured using the holdout test set.

A model must:

• maintain reasonable performance
• not degrade drastically
• not depend on specific records

If the model only works on known data, it is rejected.

---

9. Model Comparison

After evaluation, the module compares all candidates using:

• performance
• stability
• generalization
• interpretability

The highest score alone does not win.

The selected model is the **most reliable model**, not the most complex.

---

10. Output Written to Global Analysis Ledger

The module writes the **Model Evaluation Record**, containing:

Training Results
Performance metrics

Overfitting Assessment
Whether memorization occurred

Stability Assessment
Sensitivity to variation

Generalization Results
Behavior on unseen data

Selected Model
Chosen candidate and reasoning

Rejected Models
Why they were rejected

---

11. Operational Rules

1. A model cannot be selected using performance score alone.
2. The holdout test set must not be used during training.
3. All evaluations must be recorded.
4. Unstable models must be rejected.
5. A simpler reliable model may be preferred.

---

12. Relationship to Other Modules

Model Candidate Generator
→ provides candidates

Reliability & Validation Module
→ verifies reasoning alignment

Prediction & Monitoring Module
→ deploys final model

Dashboard Composer
→ displays high-risk cases

Report Generator
→ explains model behavior

---

13. Why This Module Is Critical

Most systems evaluate models mathematically.

EVA evaluates models **behaviorally**.

A model in EVA is not considered good because it predicts well once.

It is considered good because it behaves consistently and responsibly across situations.

The Training & Evaluation Module ensures EVA never deploys a model that cannot be trusted in real-world decision making.

Reliability & Validation (Explainability) Module

📄 EVA System Module Specification

Reliability & Validation (Explainability) Module (RVEM)

---

1. Purpose

The **Reliability & Validation (Explainability) Module (RVEM)** verifies that the selected model behaves in a logically consistent and explainable manner relative to the dataset and previously established analytical reasoning.

The module ensures the model's decisions are aligned with:

• observed data patterns
• generated hypotheses
• real-world interpretation of features

The goal is not only to know that the model predicts correctly, but to understand *why* it predicts correctly.

---

2. Design Philosophy

A model that cannot be explained cannot be trusted.

A model may achieve high predictive performance while relying on:

• irrelevant variables
• indirect leakage
• unstable correlations

Such models produce unreliable real-world decisions.

Therefore EVA requires that:

model behavior must agree with evidence derived from the dataset.

The model is treated as a hypothesis tester, not an authority.

---

3. Inputs

The module reads from the Global Analysis Ledger:

• selected model
• feature set used for training
• evaluation results
• hypotheses from Investigation & Hypothesis Engine
• feature reasoning from Feature Intelligence Engine
• dataset identity and domain

---

4. Feature Influence Analysis

The module determines which variables most influenced the model's predictions.

It identifies:

• important features
• direction of influence
• magnitude of influence

The goal is to understand how the model is making decisions.

---

5. Hypothesis Consistency Check

The module compares model behavior with earlier analytical findings.

For each major hypothesis:

The module checks whether the model supports or contradicts it.

Examples:

If earlier analysis found inactivity increases churn
→ model should rely on inactivity-related features

If model ignores key evidence
→ possible reasoning failure

A contradiction does not automatically invalidate the model, but it must be explained.

---

6. Leakage & Proxy Detection

The module inspects whether the model is indirectly using forbidden information.

This includes:

• encoded identifiers
• near-duplicate predictors
• post-outcome variables
• proxies strongly tied to the target

If detected, the model is rejected and sent back to earlier stages.

---

7. Sensitivity Testing

The module tests model behavior under small controlled input changes.

It evaluates:

• whether predictions change reasonably
• whether small irrelevant changes cause large prediction shifts

Unstable behavior indicates unreliable reasoning.

---

8. Case-Level Explanation

For selected records, the module generates case-level explanations:

• why a prediction was made
• which features contributed most
• how strong each influence was

This enables the dashboard and report to explain individual outcomes.

---

9. Confidence Assessment

The module evaluates confidence reliability.

Not all predictions should be treated equally.

The module identifies:

• high-confidence predictions
• uncertain predictions
• borderline cases

Uncertain predictions are flagged for caution.

---

10. Validation Outcome

The module assigns one of three statuses:

Validated
Model reasoning aligns with evidence

Conditionally Validated
Usable but requires caution

Rejected
Reasoning unreliable or inconsistent

Rejected models are not deployed.

---

11. Output Written to Global Analysis Ledger

The module writes the **Model Validation Record**, containing:

Feature Influence Summary
What drove predictions

Hypothesis Alignment
Agreement with analytical findings

Leakage Inspection
Whether improper signals exist

Sensitivity Behavior
Stability of predictions

Prediction Confidence Profile
Reliability levels

Validation Status
Validated / Conditional / Rejected

---

12. Operational Rules

1. A model cannot be deployed without validation.
2. Contradictions must be documented.
3. Leakage results in rejection.
4. Predictions must be explainable.
5. Confidence must be attached to predictions.

---

13. Relationship to Other Modules

Training & Evaluation Module
→ provides selected model

Prediction & Monitoring Module
→ deploys validated model

Dashboard Composer
→ displays case explanations

Report Generator
→ communicates reasoning

---

14. Why This Module Is Critical

Traditional pipelines stop at accuracy.

EVA continues to reasoning.

The Reliability & Validation Module ensures EVA never presents a model as intelligent unless its behavior is understandable and grounded in the data.

This module converts machine learning from a black box into a justified decision-support system.

Prediction & Monitoring + Dockerized Deployment

📄 EVA System Module Specification

Prediction, Monitoring & Dockerized Deployment Module (PMDD)

---

1. Purpose

The **Prediction, Monitoring & Dockerized Deployment Module (PMDD)** converts a validated model into a reusable prediction service and continuously evaluates whether its predictions remain reliable over time.

This module allows EVA to move from analysis to operational assistance.

The model is no longer a one-time result.
It becomes an ongoing decision-support component.

---

2. Design Philosophy

A trained model is not a finished product.

Reality changes:
• user behavior changes
• market conditions shift
• measurement patterns drift

A model that was once correct can gradually become misleading.

Therefore EVA does not simply deploy models.
It supervises them.

The module treats the model as a monitored instrument, not a permanent truth source.

---

3. Preconditions

The module activates only if:

• a model has passed the Reliability & Validation (Explainability) Module
• the validation status is **Validated** or **Conditionally Validated**

Rejected models cannot be deployed.

---

4. Dockerized Model Service

The selected model is packaged as an isolated service.

The purpose of containerization is:

• reproducibility
• independence from the main EVA system
• consistent behavior across machines
• safe execution

The container contains:

• trained model
• feature schema
• preprocessing rules
• prediction logic
• explanation interface

The model must never rely on external state.

---

5. Prediction Interface

The deployed service accepts new input records that follow the learning view schema.

For each record, the system returns:

• predicted outcome
• confidence estimate
• risk category
• explanation reference

Predictions without confidence values are not permitted.

---

6. Feature Consistency Check

Before generating a prediction, the module verifies:

• required features are present
• feature formats are valid
• values fall within expected ranges

If input is inconsistent, the system must:

• reject prediction
or
• flag the result as unreliable

---

7. Prediction Logging

Every prediction request is recorded.

The log includes:

• timestamp
• input summary
• prediction
• confidence level

This allows later review and monitoring.

---

8. Monitoring Responsibilities

The module continuously checks for model degradation.

It monitors:

Data Drift
Input distributions changing from training data

Confidence Drift
Predictions becoming uncertain

Behavior Drift
Model predictions changing patterns

If drift is detected, the module alerts EVA.

---

9. Retraining Trigger

The module recommends retraining when:

• prediction confidence declines significantly
• input patterns differ substantially from training data
• error patterns emerge in monitored outcomes

The system does not automatically retrain without review.

It requests a new analysis session.

---

10. Integration with Dashboard

The Dashboard Composer receives:

• high-risk predictions
• uncertain cases
• drift alerts

The dashboard must visibly indicate when model reliability is weakening.

---

11. Integration with Report Generator

The Report Generator includes:

• how predictions are produced
• reliability limitations
• monitoring status

The report must never present predictions without explaining uncertainty.

---

12. Output Written to Global Analysis Ledger

The module writes the **Prediction Deployment Record**, including:

Deployment Status
Model active or inactive

Prediction Schema
Required inputs

Confidence Behavior
Typical prediction reliability

Monitoring Results
Detected drift or stability

Retraining Recommendation
If needed

---

13. Operational Rules

1. Only validated models may be deployed.
2. Every prediction must include confidence.
3. Predictions must be logged.
4. Drift must be monitored.
5. Users must be warned when reliability declines.

---

14. System Interaction

Reliability & Validation Module
→ approves model

Dashboard Composer
→ displays predictions and alerts

Report Generator
→ communicates prediction meaning

Global Analysis Ledger
→ stores monitoring history

---

15. Why This Module Is Critical

Most ML systems end after training.

EVA continues after deployment.

The model becomes a continuously supervised assistant rather than a static artifact.

This module allows EVA to answer not only:

“what is happening”
and
“what will happen”

but also:

“Can we still trust this prediction?”